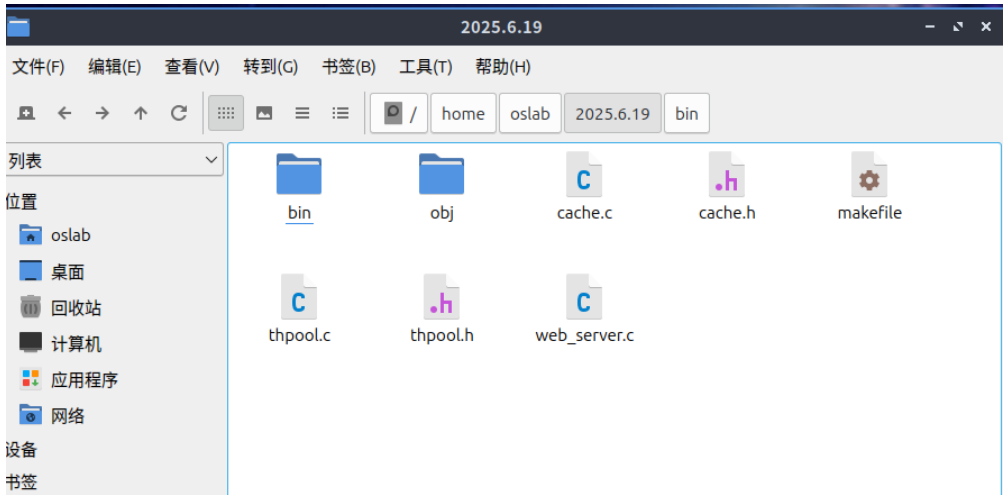


带缓存和线程池的 Web 服务器

代码目录：



ab 测试结果：

```
oslab@oslab-vmware: ~/2025.6.19/bin x
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 127.0.0.1 (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests
Completed 1000 requests
Finished 1000 requests

Server Software:
Server Hostname: 127.0.0.1
Server Port: 8080

Document Path: /index.html
Document Length: 48 bytes

Concurrency Level: 10
Time taken for tests: 0.072 seconds
Complete requests: 1000
Failed requests: 0
Non-2xx responses: 1000
Total transferred: 99000 bytes
HTML transferred: 48000 bytes
Requests per second: 13870.78 [#/sec] (mean)
Time per request: 0.721 [ms] (mean)
Time per request: 0.072 [ms] (mean, across all concurrent requests)
Transfer rate: 1341.02 [Kbytes/sec] received

Connection Times (ms)
      min      mean[+/-sd] median   max
Connect:    0      0   0.1      0      1
Processing:  0      0   0.1      0      1
Waiting:    0      0   0.1      0      1
Total:       0      1   0.1      1      2

Percentage of the requests served within a certain time (ms)
 50%    1
 66%    1
 75%    1
 80%    1
 90%    1
 95%    1
 98%    1
```

QPS (每秒请求数): Requests per second: 13870.78 [#/sec]
(mean)

平均响应时间:

Time per request: 0.721 [ms] (mean)

Time per request: 0.072 [ms] (mean, across all
concurrent requests)

响应时间优化: 缓存减少了磁盘 IO 开销, 热点资源直接从内存
读取。

QPS 提升: 并发处理能力增强, 服务器可同时处理更多请求。

Web_server 服务器代码:

```
// web_server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <pthread.h>
#include <time.h>
#include <sys/stat.h>
#include <fcntl.h>
#include "thpool.h"
#include "cache.h"

#define PORT 8080
#define MAX_CLIENTS 100
#define BUFFER_SIZE 4096
#define DOC_ROOT "./www"
#define LOG_FILE "./www/access.log"

// 日志文件描述符
int log_fd = -1;

// 客户端请求结构体
```

```

typedef struct {
    int client_fd;
    Cache *cache;
} request_data;

// 写入日志
void write_log(const char *message) {
    if (log_fd == -1) return;

    time_t now = time(NULL);
    char time_str[26];
    struct tm *tm_info = localtime(&now);

    strftime(time_str, 26, "%Y-%m-%d %H:%M:%S", tm_info);
    char log_line[1024];

    snprintf(log_line, sizeof(log_line), "[%s] %s\n", time_str,
message);
    write(log_fd, log_line, strlen(log_line));
}

// 处理 HTTP 请求
void handle_request(void *arg) {
    request_data *data = (request_data *)arg;
    int client_fd = data->client_fd;
    Cache *cache = data->cache;
    free(data);

    char buffer[BUFFER_SIZE] = {0};
    char filename[BUFFER_SIZE] = {0};
    char *file_content = NULL;
    size_t file_size = 0;
    struct timespec start, end;
    long long response_time = 0;

    // 记录开始时间
    clock_gettime(CLOCK_MONOTONIC, &start);

    // 读取 HTTP 请求
    int bytes_read = read(client_fd, buffer, BUFFER_SIZE - 1);
    if (bytes_read <= 0) {
        close(client_fd);
        return;
    }
}

```

```

// 解析请求获取文件名
char *method = strtok(buffer, " ");
if (method && strcmp(method, "GET") == 0) {
    char *path = strtok(NULL, " ");
    if (path) {
        // 处理根路径请求
        if (strcmp(path, "/") == 0) {
            strcpy(filename, DOC_ROOT "/index.html");
        } else {
            snprintf(filename, BUFFER_SIZE, "%s%s", DOC_ROOT,
path);
        }

        // 检查文件是否存在
        FILE *file = fopen(filename, "rb");
        if (file) {
            fseek(file, 0, SEEK_END);
            file_size = ftell(file);
            fseek(file, 0, SEEK_SET);

            // 尝试从缓存获取
            file_content = get_from_cache(cache, filename,
&file_size);
            if (file_content) {
                // 缓存命中
                char log_msg[512];
                snprintf(log_msg, sizeof(log_msg), "CACHE_HIT
- %s", filename);
                write_log(log_msg);
            } else {
                // 缓存未命中, 从磁盘读取
                char log_msg[512];
                snprintf(log_msg, sizeof(log_msg), "CACHE_MISS
- %s", filename);
                write_log(log_msg);

                file_content = (char *)malloc(file_size + 1);
                if (file_content) {
                    fread(file_content, 1, file_size, file);
                    file_content[file_size] = '\0';
                    // 添加到缓存
                    add_to_cache(cache, filename, file_content,
file_size);

```

```

        }
    }
    fclose(file);
} else {
    // 文件不存在, 返回 404
    file_size = strlen("HTTP/1.1 404 Not Found\r\nContent-
Type: text/html\r\n\r\n<html><body><h1>404 Not
Found</h1></body></html>");
    file_content = "HTTP/1.1 404 Not Found\r\nContent-Type:
text/html\r\n\r\n<html><body><h1>404 Not Found</h1></body></html>";

    char log_msg[512];
    snprintf(log_msg, sizeof(log_msg), "ERROR - File Not
Found: %s", filename);
    write_log(log_msg);
}
}

// 构建 HTTP 响应
if (file_content) {
    char response[BUFFER_SIZE + file_size];
    if (strstr(file_content, "404 Not Found")) {
        snprintf(response, BUFFER_SIZE + file_size, "%s",
file_content);
    } else {
        snprintf(response, BUFFER_SIZE + file_size,
"HTTP/1.1 200 OK\r\n"
"Content-Type: text/html\r\n"
"Content-Length: %zu\r\n"
"\r\n%s",
file_size, file_content);
    }

    // 发送响应
    write(client_fd, response, strlen(response));

    // 释放资源
    if (file_content && !strstr(file_content, "404 Not Found")) {
        // 注意: 缓存中的内容由缓存模块管理, 这里不释放
        file_content = NULL;
    }
}
}

```

```

// 记录结束时间并计算响应时间
clock_gettime(CLOCK_MONOTONIC, &end);
response_time = (end.tv_sec - start.tv_sec) * 1000 +
                (end.tv_nsec - start.tv_nsec) / 1000000;

// 记录响应时间日志
char log_msg[512];
snprintf(log_msg, sizeof(log_msg), "RESPONSE_TIME - %s - %lld ms",
filename, response_time);
write_log(log_msg);

close(client_fd);
}

int main() {
// 创建日志目录
mkdir("./www", 0755);

// 打开日志文件
log_fd = open(LOG_FILE, O_WRONLY | O_CREAT | O_APPEND, 0644);
if (log_fd == -1) {
    printf("Failed to open log file: %s\n", LOG_FILE);
    // 继续执行，不影响服务器主要功能
} else {
    write_log("Server started");
}

// 初始化缓存
Cache *cache = init_cache(50);
if (!cache) {
    printf("Failed to initialize cache\n");
    if (log_fd != -1) close(log_fd);
    return 1;
}

// 创建线程池
thread_pool *pool = thpool_init(8, 100);
if (!pool) {
    printf("Failed to initialize thread pool\n");
    destroy_cache(cache);
    if (log_fd != -1) close(log_fd);
    return 1;
}

```

```

// 创建套接字
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd < 0) {
    printf("Failed to create socket\n");
    thpool_destroy(pool);
    destroy_cache(cache);
    if (log_fd != -1) close(log_fd);
    return 1;
}

// 设置套接字选项
int opt = 1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
sizeof(opt)) < 0) {
    printf("Failed to set socket options\n");
    close(server_fd);
    thpool_destroy(pool);
    destroy_cache(cache);
    if (log_fd != -1) close(log_fd);
    return 1;
}

// 绑定地址
struct sockaddr_in address;
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

if (bind(server_fd, (struct sockaddr *)&address, sizeof(address))
< 0) {
    printf("Failed to bind\n");
    close(server_fd);
    thpool_destroy(pool);
    destroy_cache(cache);
    if (log_fd != -1) close(log_fd);
    return 1;
}

// 监听连接
if (listen(server_fd, MAX_CLIENTS) < 0) {
    printf("Failed to listen\n");
    close(server_fd);
    thpool_destroy(pool);
    destroy_cache(cache);
}

```

```

        if (log_fd != -1) close(log_fd);
        return 1;
    }

    printf("Web server running on port %d\n", PORT);

    // 接受客户端连接
    while (1) {
        struct sockaddr_in client_address;
        socklen_t client_len = sizeof(client_address);
        int client_fd = accept(server_fd, (struct sockaddr
*)&client_address, &client_len);

        if (client_fd < 0) {
            printf("Failed to accept connection\n");
            continue;
        }

        // 创建请求数据并添加到线程池
        request_data *data = (request_data
*)malloc(sizeof(request_data));
        if (data) {
            data->client_fd = client_fd;
            data->cache = cache;
            thpool_add_work(pool, handle_request, (void *)data);
        } else {
            close(client_fd);
        }
    }

    // 清理资源
    close(server_fd);
    thpool_destroy(pool);
    destroy_cache(cache);
    if (log_fd != -1) close(log_fd);

    return 0;
}

```